IF3170: Inteligensi Artifisial

Laporan Tugas Besar 1

Implementasi Pemilihan dan Penataan Paket dalam Truk Menggunakan Algoritma Local Search



Dipersiapkan oleh:

13524074	Bernhard Aprillio Pramana
13524110	Jennifer Khang
13524122	Nathaniel Christian

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
JL. GANESA 10, BANDUNG 40132
2025

Daftar Isi

Daftar Isi **2**
Daftar Tabel **4**
Daftar Gambar **5**
Tabel Kontribusi **6**
Pendahuluan **7**
1.1 Deskripsi Persoalan 7
1.1.1 Variabel Permasalahan 7
1.1.2 Representasi State 8
1.1.3 Objective Function 8
1.1.4 Constraints 8
1.1.5 Output 8
Pembahasan Solusi **9**
2.1 Struktur Codebase 9
2.2 Model Domain 10
2.2.1 Struktur data Package 10
2.2.2 Struktur data Truck 10
2.2.3 Struktur data Problem 10
2.2.4 Struktur data State 11
2.2.5 Struktur data Result 11
2.3 Evaluator 12
2.4 Algoritma dan Implementasi Local Search 12
2.3.1 Steepest-Ascent Hill-Climbing 13
2.3.2 Simulated Annealing 14
2.3.3 Genetic Algorithm 18
2.3.4 Algoritma Bonus 22
2.3.4.1 Algoritma Bonus Hill Climbing with Sideways Move 22
2.3.4.1 Algoritma Bonus Hill Climbing 24
2.5.1 Hasil Eksperimen 27
2.5.1.1 Eksperimen Steepest-Ascent Hill Climbing 27
2.5.1.2 Eksperimen Simulated Annealing 29
2.5.1.3 Eksperimen Genetic Algorithm 31
2.5.1.4 Eksperimen Hill Climbing with Sideways Move 35
2.5.1.5 Eksperimen Random Restart Hill Climbing 37
2.5.2 Pembahasan 39
Kesimpulan dan Saran **42**
4.1 Kesimpulan 42
Daftar Pustaka **43**
Pernyataan Penggunaan AI **44**

Daftar Tabel

Tabel 1.1 Tabel Daftar Variabel Permasalahan.....7
Tabel 2.5.1.1.1 Tabel Hasil Eksperimen Steepest-Ascent Hill Climbing..... 27
Tabel 2.5.1.2.1 Tabel Hasil Eksperimen Simulated Annealing.....29
Tabel 2.5.1.3.1 Tabel Hasil Eksperimen Genetic Algorithm..... 32
Tabel 2.5.1.4.1 Tabel Hasil Eksperimen Hill Climbing with Sideways Move..... 35
Tabel 2.5.1.5.1 Tabel Hasil Eksperimen Random Restart Hill Climbing..... 37

Daftar Gambar

Gambar 2.4.1 Visualiser untuk Initial dan Final State.....26
Gambar 2.4.2 Visualiser dan GUI.....27
Gambar 2.5.1.1.1 Gambar plot value Steepest-Ascent Hill Climbing (gambar bukan hasil dari program melainkan secara manual menggabungkan ke-3 run).....28
Gambar 2.5.1.1.2 Gambar visualisasi Steepest-Ascent Hill Climbing run-1.....29
Gambar 2.5.1.1.2 Gambar visualisasi Steepest-Ascent Hill Climbing run-2.....29
Gambar 2.5.1.1.2 Gambar visualisasi Steepest-Ascent Hill Climbing run-3.....29
Gambar 2.5.1.2.1 Gambar plot value & frekuensi stuck Simulated Annealing seed 100.....30
Gambar 2.5.1.2.2 Gambar plot value & frekuensi stuck Simulated Annealing seed 101.....30
Gambar 2.5.1.2.3 Gambar plot value & frekuensi stuck Simulated Annealing seed 102.....31
Gambar 2.5.1.2.4 Gambar visualisasi Simulated Annealing run ke-1.....31
Gambar 2.5.1.2.5 Gambar visualisasi Simulated Annealing run ke-2.....31
Gambar 2.5.1.2.6 Gambar visualisasi Simulated Annealing run ke-3.....31
Gambar 2.5.1.3.1 Gambar plot value Genetic Algorithm Konfigurasi 1.....33
Gambar 2.5.1.3.2 Gambar plot value Genetic Algorithm Konfigurasi 2.....33
Gambar 2.5.1.3.3 Gambar plot value Genetic Algorithm Konfigurasi 3.....34
Gambar 2.5.1.3.4 Gambar plot value Genetic Algorithm Konfigurasi 4.....34
Gambar 2.5.1.3.5 Gambar plot value Genetic Algorithm Konfigurasi 5.....35
Gambar 2.5.1.3.6 Gambar plot value Genetic Algorithm Konfigurasi 6.....35
Gambar 2.5.1.4.1 Gambar plot value Hill Climbing with Sideways Move run ke-1.....36
Gambar 2.5.1.4.2 Gambar plot value Hill Climbing with Sideways Move run ke-2.....36
Gambar 2.5.1.4.3 Gambar plot value Hill Climbing with Sideways Move run ke-3.....37
Gambar 2.5.1.4.4 Gambar visualisasi Hill Climbing with Sideways Move run-1.....37
Gambar 2.5.1.4.5 Gambar visualisasi Hill Climbing with Sideways Move run-2.....37
Gambar 2.5.1.4.6 Gambar visualisasi Hill Climbing with Sideways Move run-3.....37
Gambar 2.5.1.5.1 Gambar plot value Random Restart Hill Climbing run ke-1.....38
Gambar 2.5.1.5.2 Gambar plot value Random Restart Hill Climbing run ke-2.....38
Gambar 2.5.1.5.3 Gambar plot value Random Restart Hill Climbing run ke-3.....39
Gambar 2.5.1.5.4 Gambar visualisasi Random Restart Hill Climbing run-1.....39
Gambar 2.5.1.5.5 Gambar visualisasi Random Restart Hill Climbing run-2.....39
Gambar 2.5.1.5.6 Gambar visualisasi Random Restart Hill Climbing run-3.....39

Tabel Kontribusi

NIM	Nama	Kontribusi
13524074	Bernhard Aprillio Pramana	❖ Laporan ❖ Implementasi parser dan visualizer ❖ Implementasi Simulated Annealing
13524110	Jennifer Khang	❖ Laporan ❖ Implementasi Genetic Algorithm dan utils ❖ Implementasi Algoritma bonus ❖ Melakukan testing dan perbaikan
13524122	Nathaniel Christian	❖ Laporan ❖ Implementasi model awal dan utils ❖ Implementasi Hill Climbing ❖ Melakukan testing dan perbaikan

Bab 1
Pendahuluan

1.1 Deskripsi Persoalan

Permasalahan tugas besar ini merupakan variasi dari knapsack problem tiga dimensi. Program harus bisa menentukan konfigurasi pengisian paket ke dalam sebuah truk peti kemas dengan memaksimalkan total nilai paket yang diangkut. Setiap paket memiliki identitas, ukuran, orientasi, nilai, berat, dan status mudah pecah. Sebuah paket boleh berada di dalam atau di luar truk. Paket yang berada di dalam truk harus menempati sel-sel grid secara utuh.

Koordinat dalam ruang truk direpresentasikan sebagai sebuah grid tiga dimensi dengan urutan indeks $grid[z][y][x]$. Posisi paket adalah koordinat sudut kiri-bawah-belakangnya, sedangkan ukuran efektif paket ditentukan oleh orientasinya. Karena sebuah balok memiliki enam permutasi orientasi sumbu, rotasi dapat mengubah ukuran efektif pada sumbu x, y, dan z.

1.1.1 Variabel Permasalahan

Tabel 1.1 Tabel Daftar Variabel Permasalahan

Entitas	Atribut	Contoh	Keterangan
Package	id	“RusdiPackage”	*Identifier*
	Dimensions	{w:3, l:4, h:1}	Ukuran sebuah paket, yaitu *width*, *length*, dan *height*
	Orientation	{w:x, l:y, h:z}	Sumbu yang sejajar dengan setiap sisi paket. Terdapat enam kemungkinan orientasi, misalnya {w:y, l:x, h:z} setelah rotasi 90° terhadap sumbu *z*.
	Value	60	Nilai harga sebuah paket (satuan unit)
	Weight	5	Berat paket (satuan unit berat)
	IsFragile	False	Benda pecah-belah
	ETA	3	Estimated Time of Arrival, perkiraan waktu kedatangan paket (dalam sisa hari)
Truck	Dimensions	{w:10, l:20, h:5}	Ukuran sebuah peti kemas pada truk, yaitu *width*, *length*, dan *height*
	MaxCapacity	500	Kapasitas angkut maksimum pada suatu peti kemas (satuan unit berat)

1.1.2 Representasi State

Secara umum, sebuah state adalah konfigurasi sementara yang menjelaskan keadaan seluruh paket dan truk pada satu titik selama proses pencarian. State harus dapat membedakan paket yang berada di dalam truk dan paket yang berada di luar truk, serta menyimpan informasi yang diperlukan untuk menentukan nilai objective function.

Sebuah algoritma local search menghasilkan state baru dengan menerapkan operasi neighbor, misalnya memindahkan paket, menukar dua paket, atau mengubah orientasi paket. State awal dibangkitkan secara acak, sedangkan state akhir adalah konfigurasi terbaik yang ditemukan algoritma berdasarkan objective function. Detail mengenai implementasi struktur data state akan dijelaskan pada Bab 2.

1.1.3 Objective Function

Objective function yang digunakan adalah total dari seluruh *value* paket yang berada di dalam truk.

1.1.4 Constraints

Sebuah state dikatakan valid apabila memenuhi seluruh constraint berikut.

- Tidak menembus dinding truk. Semua sel paket harus berada pada rentang ukuran grid truk.
- Satu sel grid tidak boleh berisi lebih dari satu paket.
- Tidak melayang. Paket yang tidak berada pada lantai $z = 0$ harus mempunyai paling sedikit satu sel grid alas yang memperoleh support dari paket lain tepat di bawahnya.
- Paket yang *fragile* tidak boleh menjadi support.
- Total berat paket di dalam truk tidak boleh melebihi $maxCapacity$.

1.1.5 Output

Output persoalan ini adalah konfigurasi tata letak terbaik yang ditemukan oleh algoritma *local search*, serta visualisasi *initial state* dan *final state*-nya.

1.2 Asumsi Perancangan dan Implementasi

Sistem dirancang dengan beberapa asumsi, yakni sebagai berikut.

- Dalam abstraksi model persoalan, tim menganggap bahwa paket boleh menyentuh sisi.
- Untuk kebutuhan plot Simulated Annealing, frekuensi *stuck* bertambah jika sebuah state tidak berubah dari state originalnya.
- Dalam merancang Genetic Algorithm, tim awalnya memakai mekanisme turnamen dan elitism. Namun, tim baru sadar akan keperluan tugas besar untuk mengimplementasikannya dalam ‘salindia kuliah‘, sehingga optimasi yang dilakukan sebelumnya terpaksa tidak dimasukkan.

Bab 2

Pembahasan Solusi

2.1 Struktur Codebase

Implementasi solusi menggunakan bahasa pemrograman Python. Struktur modul utama adalah sebagai berikut.

```
IF3170_G03_nAthanIntelligence/
├── README.md
├── pyproject.toml
├── uv.lock
├── .python-version
├── .gitignore
│
├── src/
│   ├── main.py                         # Membaca input, run algoritma, dan menyimpan hasil
│   ├── visualizer.py                   # Visualisasi state atau run algoritma tervisualized
│   │
│   ├── algo/                           # Algoritma
│   │   ├── hill_climb.py               # Steepest ascent hill climbing
│   │   ├── hill_climb_sideway.py
│   │   ├── hill_climb_random_restart.py
│   │   ├── simulated_annealing.py      # Simulated annealing
│   │   └── genetic.py                  # Genetic algorithm
│   │
│   ├── models/                         # Struktur data
│   │   ├── problem.py                  # Representasi problem
│   │   ├── package.py                  # Representasi package
│   │   ├── truck.py                    # Representasi truck
│   │   ├── state.py                    # Representasi sebuah state
│   │   ├── result.py                   # Bentuk output/hasil pencarian
│   │   └── types/
│   │       ├── dims.py                 # Dimensi: w, l, h
│   │       ├── position.py             # Koordinat: x, y, z
│   │       └── orientation.py          # Orientasi paket terhadap sumbu
│   │
│   └── utils/                          # Fungsi pendukung
│       ├── parser.py                   # Parsing JSON menjadi Problem dan resolusi file
│       ├── evaluate.py                 # Nilai objektif dan validasi penempatan
│       ├── move.py                     # Place, move, rotate, swap, remove, neighbors
│       ├── random_state.py             # Pembuatan posisi dan state awal acak
│       └── plotting.py                 # Grafik statistik algoritma
│
├── test/
│   ├── input/
│   │   ├── test-1.json
│   │   ├── test-2.json
│   │   ├── test-3.json
│   │   ├── test-4.json
│   │   └── test-5.json
│   └── scripts/
│       ├── test_parser.py
│       ├── test_evaluate.py
│       ├── test_hill_climb.py
│       ├── test_simulated_annealing.py
│       ├── test_genetic.py
│       └── experiments/
│           └── run_genetic.py
│
├── output/                             # Penyimpanan hasil
└── docs/                               # Folder dokumentasi
```

2.2 Model Domain

Untuk abstraksi dari permasalahan, kami mendefinisikan beberapa tipe yang menjadi basis implementasi.

Tipe	Fungsi
Dimension	Menyimpan ukuran w, l, dan h, serta memvalidasi bahwa dimensinya integer positif.
Orientation	Menyimpan orientasi atau rotasi paket di dalam koordinat truk.
Position	Merepresentasikan suatu titik koordinat di dalam grid menggunakan x, y, dan z.

Selain itu, ada beberapa model struktur data yang diimplementasikan.

2.2.1 Struktur data Package

src/models/package.py

```
class Package:
    id: str
    dimensions: Dimensions
    orientation: Orientation
    value: int
    weight: float
    isFragile: bool
    eta: int
```

Package menyimpan informasi lengkap suatu paket, yaitu ID, dimensi, orientasi awal, nilai, berat, status fragile, dan ETA.

2.2.2 Struktur data Truck

src/models/truck.py

```
class Truck:
    dimensions: Dimensions
    maxCapacity: int
```

Truck menyimpan informasi mengenai truk, yaitu dimensi ruang peti kemas dan kapasitas berat maksimum. *maxCapacity* digunakan oleh evaluator untuk memastikan total berat paket yang berada di dalam truk tidak melebihi kapasitas.

2.2.3 Struktur data Problem

src/models/problem.py

```
class Problem:
    packages: dict[str, Package]  # packages catalog
```

```
    truck: Truck
```

Problem merepresentasikan satu permasalahan yang terdiri atas kumpulan paket dan satu truk. Katalog paket disimpan dalam bentuk dictionary dengan package ID sebagai key.

2.2.4 Struktur data State

src/models/state.py

```
# define cell and grid types
Cell = str | None  # None means empty; ini isinya package id
Grid = list[list[list[Cell]]]  # [z][y][x]

# define state
class State:
    problem: Problem
    grid: Grid
    pkg_orientation: dict[str, Orientation]
    out_pkg: set[str]
    value: int  # acts like a cache; need to be recalculated
```

Cell merupakan tipe alias untuk isi satu sel pada grid. Nilai string menyimpan ID paket yang menempati sel tersebut, sedangkan None menunjukkan bahwa sel masih kosong.

Grid merupakan tipe alias untuk struktur grid tiga dimensi. Urutan indeksnya adalah $grid[z][y][x]$, sehingga grid hanya merepresentasikan ruang di dalam truk. Setiap paket yang berada di luar truk tidak disimpan di dalam grid.

State merepresentasikan satu konfigurasi solusi pada proses pencarian. State menyimpan sebuah Problem, isi grid, orientasi setiap paket melalui pkg_orientation, himpunan paket di luar truk melalui out_pkg, dan nilai objective function melalui value.

2.2.5 Struktur data Result

src/models/result.py

```
class Result:
    initial_state: State
    state: State
    value_history: list[int]
    iterations: int
    duration_seconds: float
    metrics: dict[str, object] = field(default_factory=dict)
```

Result menyimpan hasil eksekusi suatu algoritma pencarian. Informasi yang disimpan mencakup state awal, state akhir, riwayat nilai objective function, jumlah iterasi, durasi pencarian, serta metrik tambahan seperti jumlah neighbor yang dievaluasi atau jumlah accepted move.

2.3 Evaluator

src/utils/evaluate.py

```
class Evaluate:
    @staticmethod
    def value(state: State) -> int:
        return sum(
            placed[0].value
            for placed in _placed_packages(state)
        )

    @staticmethod
    def is_valid(state: State) -> bool:
        return (
            Evaluate.inside_truck(state)
            and Evaluate.no_floating(state)
            and Evaluate.fragile_not_supporting(state)
            and Evaluate.within_capacity(state)
        )
```

File ini menyediakan fungsi evaluator atau *objective function* agar seluruh algoritma menggunakan aturan validitas yang sama, sehingga hasil yang didapatkan selalu sesuai constraint. Fungsi Evaluate.value() hanya menjumlahkan nilai paket yang terdeteksi pada grid. Fungsi Evaluate.is_valid() menggabungkan pemeriksaan geometri, support, fragile, dan kapasitas.

2.4 Algoritma dan Implementasi Local Search

Local Search adalah algoritma pencarian yang dirancang untuk situasi di mana jalur untuk mencapai tujuan tidak relevan. Algoritma ini berfokus pada penemuan keadaan atau konfigurasi akhir (final state) yang paling optimal sebagai solusinya. Local Search hanya menyimpan satu keadaan saat ini (single current state).

Berbeda dengan classical search, pada local search path cost tidak ada. Pencarian dievaluasi berdasarkan state value, yang ditentukan oleh fungsi objektif (objective function) atau fungsi biaya heuristik (heuristic cost function). Local Search tidak secara eksplisit menguji apakah suatu tujuan telah tercapai, namun mencari nilai keadaan yang maksimal (maximum state value). Konfigurasi dalam Local Search selalu dalam bentuk keadaan yang lengkap (complete configurations), tidak seperti classical search yang incremental.

Local search memiliki tiga komponen utama, State merupakan kondisi dalam pencarian, didefinisikan sebagai konfigurasi yang lengkap. Neighbor atau Successor merupakan state-state lain yang dapat dicapai secara langsung dari current state. Aksi dalam algoritma ini didefinisikan sebagai pergerakan dari current state ke neighbor-nya untuk mencari state terbaik (global optimum).

Berdasarkan konsep State dan pergerakan ke Neighbor (Action: move to neighbors) untuk menemukan global optimum, terdapat beberapa algoritma spesifik yang merupakan penerapan dari Local Search, di antaranya adalah sebagai berikut.

2.3.1 Steepest-Ascent Hill-Climbing

Steepest-Ascent Hill Climbing adalah algoritma local search yang memulai pencarian dari satu state, kemudian secara berulang memeriksa neighbor di sekitar state tersebut. Hill Climbing dengan varian steepest-ascent memilih neighbor dengan kenaikan nilai objective function paling besar.

Pada setiap iterasi, program membangkitkan kandidat yang dianggap berpotensi meningkatkan nilai. Kandidat dengan nilai terbesar dipilih apabila nilainya lebih tinggi daripada state sekarang (jika nilainya sama, tidak dipilih). Pencarian berhenti ketika tidak ada kandidat yang meningkatkan objective function.

src/algo/hill_climb.py

```
def steepest_ascent_hill_climbing(...) -> Result:
    current = initial_state.copy()
    current.value = Evaluate.value(current)
    initial = current.copy()
    value_history = [current.value]
    iterations = 0

    while True:
        best_neighbor = None
        for candidate in _improving_neighbors(current):
            if candidate.value > current.value and (
                best_neighbor is None
                or candidate.value > best_neighbor.value
            ):
                best_neighbor = candidate

        iterations += 1
        if best_neighbor is None:
            value_history.append(current.value)
            break

        current = best_neighbor
        value_history.append(current.value)

    return Result(
        initial_state=initial,
        state=current.copy(),
        value_history=value_history,
        iterations=iterations,
        duration_seconds=perf_counter() - started_at,
        metrics={"termination_reason": "local_optimum"},
    )

def _improving_neighbors(state: State) -> Iterator[State]:
    # Menghasilkan neighbor valid dengan nilai objective lebih tinggi
```

Fungsi $steepest_ascent_hill_climbing()$ memilih kandidat dengan nilai terbesar dari neighbor yang dibangkitkan oleh fungsi $_improving_neighbors()$. Dengan demikian, istilah ***steepest*** berarti kandidat terbaik dipilih dari neighbor yang berhasil dibangkitkan, bukan seluruh konfigurasi state yang mungkin secara global.

2.3.2 Simulated Annealing

Simulated Annealing merupakan algoritma yang terinspirasi dari proses annealing pada material, yaitu memanaskan logam atau kaca lalu membiarkannya mendingin secara perlahan untuk menghilangkan tekanan internal dan memperkuat wujudnya. Terdapat parameter T (temperature) yang nilainya akan menurun secara bertahap seiring berjalannya iterasi pencarian.

src/algo/simulated_annealing.py

```
def simulated_annealing(
    initial_state: State,
    *,
    initial_temperature: float = 100.0,
    cooling_rate: float = 0.995,
    minimum_temperature: float = 0.1,
    max_iterations: int = 10_000,
    seed: int | None = None,
    on_iteration: Callable[[State, int], None] | None = None,
) -> Result:
    if initial_temperature <= 0:
        raise ValueError("initial_temperature harus lebih besar dari
nol")
    if not 0 < cooling_rate < 1:
        raise ValueError("cooling_rate harus berada di antara 0 dan 1")
    if minimum_temperature <= 0:
        raise ValueError("minimum_temperature harus lebih besar dari
nol")
    if max_iterations < 0:
        raise ValueError("max_iterations tidak boleh negatif")

    started_at = perf_counter()
    rng = random.Random(seed)
    current = initial_state.copy()
    if not Evaluate.is_valid(current):
        current = State.empty(initial_state.problem)
    initial = current.copy()
    best = current.copy()
    value_history = [current.value]
    temperature = initial_temperature
    accepted_moves = 0
    iterations = 0
    acceptance_probability_history: list[float | None] = []
    stuck_frequency = 0
```

```
    while iterations < max_iterations and temperature >
minimum_temperature:
        candidate = _random_neighbour(current, rng)
        probability = None
        state_changed = False
        if candidate is not None:
            delta = candidate.value - current.value
            probability = 1.0 if delta >= 0 else math.exp(delta /
temperature)
            if delta >= 0 or rng.random() < probability:
                state_changed = candidate != current
                current = candidate
                accepted_moves += 1
                if current.value > best.value:
                    best = current.copy()

        # satu iterasi tanpa perubahan state dihitung satu kali stuck
        if not state_changed:
            stuck_frequency += 1

        acceptance_probability_history.append(probability)

        # sampling gagal tetap dihitung satu iterasi dan temperatur
tetap turun
        temperature *= cooling_rate
        iterations += 1
        value_history.append(current.value)
        if on_iteration is not None:
            on_iteration(current.copy(), iterations)

    return Result(
        initial_state=initial,
        state=best,
        value_history=value_history,
        iterations=iterations,
        duration_seconds=perf_counter() - started_at,
        metrics={
            "accepted_moves": accepted_moves,
            "acceptance_probability_history":
acceptance_probability_history,
            "stuck_frequency": stuck_frequency,
        },
    )
```

Pencarian didasari oleh selisih value dari neighbour (nilai yang akan dievaluasi) dengan value current (kondisi sekarang). $\Delta E = next.value - current.value$. Dengan evaluasi penerimaannya adalah sebagai berikut.

- Jika $\Delta E \geq 0$, kondisi saat ini akan diganti oleh kondisi berikutnya atau neighbour (current = next).
- Jika $\Delta E < 0$, maka akan dihitung probabilitas penerimaan dengan $e^{\Delta E/T}$ yang akan dibandingkan dengan nilai acak (0-1). Jika hasilnya lebih besar daripada

nilai acak, kondisi saat ini diganti dengan kondisi berikutnya (current = next) untuk menghindari local optima.

Algoritma menggunakan variabel state_changed untuk melacak apakah terjadi perpindahan state pada suatu iterasi. Apabila tidak ada state valid yang ditemukan atau kandidat ditolak oleh probabilitas sehingga state_changed bernilai False, maka variabel stuck_frequency akan bertambah satu. Metrik stuck ini digunakan untuk menganalisis seberapa sering algoritma mengalami kebuntuan. Nilai probabilitas penerimaan juga dicatat ke dalam acceptance_probability_history pada tiap iterasinya. Setelah melakukan evaluasi, temperatur akan diperbarui dengan rasio cooling_rate (temperature *= cooling_rate).

src/algo/simulated_annealing.py

```
def _random_neighbour(state: State, rng: random.Random) -> State |
None:
    operations = ["place"] if state.out_pkg else []
    package_ids = sorted(state.problem.packages)
    placed = sorted(state.package_ids_in_grid())
    if placed:
        operations.append("remove")
    if package_ids:
        operations.append("rotate")
    if not operations:
        return None

    for _ in range(8):
        operation = rng.choice(operations)
        if operation == "place":
            candidate = _try_place(state, rng)
        elif operation == "remove":
            candidate = try_remove(state, rng.choice(placed))
        else:
            candidate = try_rotate(
                state,
                rng.choice(package_ids),
                rng.choice(("x", "y", "z")),
            )
        if candidate is not None:
            return candidate
    return None
```

Fungsi bertugas mendeteksi dan merumuskan aksi apa saja yang valid untuk dilakukan pada kondisi truk saat ini. Jika masih ada paket di luar truk, opsi untuk place ditambahkan. Jika sudah ada paket yang tersusun di dalam grid truk, opsi untuk remove ditambahkan. Selain itu, karena paket bisa diputar baik sebelum maupun sesudah masuk ke truk, opsi rotate selalu bisa dilakukan. Setelah daftar aksi disusun, fungsi akan mencoba mengacak salah satu operasi untuk dijalankan. Pencarian kandidat tetangga ini dibatasi maksimal 8 kali percobaan. Jika dalam 8 kali percobaan acak tersebut tidak ada satu pun operasi yang menghasilkan susunan yang valid, fungsi akan langsung

mengembalikan None. Hal ini berfungsi sebagai pengaman agar algoritma tidak terjebak dalam infinite loop.

src/algo/simulated_annealing.py

```
def _try_place(state: State, rng: random.Random) -> State | None:
    return _place(state, rng.choice(sorted(state.out_pkg)), rng)

def _place(state: State, package_id: str, rng: random.Random) -> State |
None:
    package = state.problem.get_package(package_id)
    orientation = state.pkg_orientation[package_id]
    width, length, height = _oriented_dimensions(package, orientation)
    truck = state.problem.truck.dimensions
    max_x = truck.w - width
    max_y = truck.l - length
    max_z = truck.h - height
    if min(max_x, max_y, max_z) < 0:
        return None

    z_levels = {0}
    for layer_index, layer in enumerate(state.grid):
        if layer_index < max_z and any(
            cell is not None for row in layer for cell in row
        ):
            z_levels.add(layer_index + 1)
    z_levels = tuple(sorted(z_levels))

    for _ in range(24):
        position = Position(
            x=rng.randrange(max_x + 1),
            y=rng.randrange(max_y + 1),
            z=rng.choice(z_levels),
        )
        candidate = try_place(state, package_id, position)
        if candidate is not None:
            return candidate
    return None


def _oriented_dimensions(
    package: Package, orientation: Orientation
) -> tuple[int, int, int]:
    dims = [
        getattr(package.dimensions, dimension)
        for axis in ("x", "y", "z")
        for dimension in ("w", "l", "h")
        if getattr(orientation, dimension) == axis
    ]
    return dims[0], dims[1], dims[2]
```

Fungsi penyesuaian dari move.py untuk penyesuaian algoritma Simulated Annealing. Fungsi _try_place akan memilih paket yang berada di luar truk secara acak tergantung dari variabel rng. Fungsi _oriented_dimensions bertugas untuk memetakan

dimensi paket ke sumbu ruang (x, y, z) truk untuk variabel width, length, height sesuai dengan orientasi paket. Fungsi _place memiliki batas kordinat maksimal (max_x, max_y, max_z) untuk mencegah paket menembus dinding truk, dan akan langsung membatalkan proses jika ukuran paket terdeteksi melebihi batas ruang. Paket tidak ditempatkan melayang di udara secara acak, fungsi mengevaluasi isi truk dengan layer_index untuk mengumpulkan level ketinggian yang sudah terisi barang sebagai referensi heuristik pijakan (z_levels). Algoritma mengeksekusi penempatan melalui maksimal 24 iterasi percobaan untuk menebak koordinat X dan Y secara bebas, dan koordinat Z dari tuple z_levels. Jika salah satu percobaan tersebut divalidasi tidak bertabrakan dengan paket lain, susunan state baru tersebut dikembalikan sebagai kandidat tetangga yang berhasil dibangkitkan.

2.3.3 Genetic Algorithm

Genetic Algorithm adalah algoritma pencarian dan optimasi metaheuristik yang terinspirasi dari proses evolusi biologis dan seleksi alam. Berbeda dengan Hill Climbing atau Simulated Annealing yang hanya mengevaluasi satu keadaan (single current state) pada satu waktu, GA mengevaluasi sekumpulan keadaan sekaligus yang disebut populasi (population).

Dalam konteks penyelesaian masalah ini, setiap individu di dalam populasi mewakili satu variasi susunan paket-paket di dalam truk. Evolusi terjadi dari generasi ke generasi melalui tiga operator utama:

(a) selection yang akan memilih individu-individu dengan nilai fitness terbaik untuk menjadi parent,
(b) crossover menggabungkan sifat (posisi dan orientasi paket) dari dua induk untuk menghasilkan child baru, dan
(c) mutation memberikan perubahan acak pada individu untuk mencegah populasi menjadi seragam dan terjebak di local optimum.

src/algo/genetic.py

```
def genetic_algorithm(
    initial_state: State,
    *,
    population_size: int = 50,
    max_iterations: int = 100,
    seed: int | None = None,
    on_iteration: Callable[[State, int], None] | None = None,
) -> Result:

    # cek populasi
    if isinstance(population_size, bool) or not
isinstance(population_size, int):
        raise TypeError("population_size harus berupa integer")
    if population_size < 2:
        raise ValueError("population_size minimal 2")
```

```
    # cek parameter iterasi
    if isinstance(max_iterations, bool) or not
isinstance(max_iterations, int):
        raise TypeError("max_iterations harus berupa integer")
    if max_iterations < 0:
        raise ValueError("max_iterations tidak boleh negatif")

    started_at = perf_counter()
    # copy juga memeriksa invariant struktur dan pembagian paket
dalam/luar
    initial = initial_state.copy()
    if not Evaluate.is_valid(initial):
        raise ValueError("initial_state harus valid")

    rng = Random(seed)
    population = _initialize_population(initial, population_size, rng)
    initial_best = max(population, key=lambda state: state.value).copy()

    # simpen statistik
    max_value_history: list[int] = []
    mean_value_history: list[float] = []
    _record_statistics(population, max_value_history,
mean_value_history)

    # tiap generasi dibangkitkan
    for generation in range(1, max_iterations + 1):
        next_population: list[State] = []

        while len(next_population) < population_size:
            first_parent = _select_parent(population, rng)
            second_parent = _select_parent(population, rng)
            child = _crossover(first_parent, second_parent, rng)

            if rng.random() < MUTATION_RATE:
                child = _mutate(child, rng)
            next_population.append(child)

        population = next_population
        _record_statistics(population, max_value_history,
mean_value_history)
        if on_iteration is not None:
            best = max(population, key=lambda state: state.value)
            on_iteration(best.copy(), generation)

    best = max(population, key=lambda state: state.value)
    return Result(
        initial_state=initial_best,
        state=best.copy(),
        value_history=max_value_history,
        iterations=max_iterations,
        duration_seconds=perf_counter() - started_at,
        metrics={
            "population_size": population_size,
            "mean_value_history": mean_value_history,
        },
    )
```

Pada awal generasi, fungsi menerapkan algoritma yang sudah diajarkan di perkuliahan. Populasi awal merupakan hasil random yang akan terus diperbarui tiap generasinya dengan prosedur *selection*, *crossover*, dan *mutation*. Jadi, pada tiap generasi, individu baru akan terus menerus diseleksi hingga jumlah populasi terpenuhi kembali. Prosedur ini diulangi sebanyak max_iterations.

src/algo/genetic.py

```
# func: memilih satu parent dari populasi
def _select_parent(
    population: list[State],
    rng: Random,
) -> State:
    if not population:
        raise ValueError("population tidak boleh kosong")

    # fitness dicatat ulang untuk seluruh populasi sebelum seleksi
generasi ini
    weights = [state.value for state in population]
    if sum(weights) == 0:
        return rng.choice(population)
    return rng.choices(population, weights=weights, k=1)[0]
```

Fungsi ini bertugas memilih individu yang layak menjadi parent menggunakan metode pemetaan *fitness function* berdasarkan value yang dimiliki. Dari sampel tersebut, individu dipilih secara acak dan dipetakan pada *fitness function*.

src/algo/genetic.py

```
# func: membentuk State anak langsung dari dua parent
def _crossover(
    p1: State,
    p2: State,
    rng: Random,
) -> State:
    package_ids = sorted(p1.problem.packages)
    # tidak ada titik potong internal untuk kromosom kosong atau satu
gen.
    if len(package_ids) < 2:
        return p1.copy()
    cut = rng.randrange(1, len(package_ids))

    # init
    child = State.empty(p1.problem)
    proposals: list[tuple[str, Position]] = []

    # satu gen = posisi/status dalam-luar dan orientasi satu paket.
    # prefix diwarisi dari p1, suffix dari p2 pada satu titik potong.
    for index, package_id in enumerate(package_ids):
        parent = p1 if index < cut else p2
```

```
        child.pkg_orientation[package_id] = replace(
            parent.pkg_orientation[package_id]
        )

        # fiksasi parent
        position = _package_anchor(parent, package_id)

        if position is not None:
            proposals.append((package_id, position))

    # shuffle sebelum stable sort agar prioritas pada z yang sama tidak
bias ID
    rng.shuffle(proposals)
    proposals.sort(key=lambda proposal: proposal[1].z)

    # penempatan ulang sekaligus cek invarian paket
    for package_id, position in proposals:
        candidate = try_place(child, package_id, position)
        if candidate is not None:
            child = candidate

    return child
```

Fungsi crossover menggabungkan konfigurasi letak dari dua parent menjadi satu child baru. Prosesnya dimulai dengan memilih secara acak parent mana yang akan menurunkan posisi dan orientasi untuk setiap paket. Konfigurasi ini disimpan sementara ke dalam list proposals. Untuk mencegah paket tidak memiliki support saat penggabungan, semua proposals ini diurutkan berdasarkan nilai sumbu Z (ketinggian) terendah. Paket di dasar truk lalu diletakkan lebih dulu agar menjadi support bagi paket di atasnya. Setiap eksekusi peletakan ini divalidasi menggunakan try_place.

src/algo/genetic.py

```
def _mutate(state: State, rng: Random) -> State:
    # init
    original = state.copy()
    package_ids = sorted(state.problem.packages)
    if not package_ids:
        return original

    # menentukan aksi mutasi
    operations = ["move", "rotate"]
    if len(package_ids) >= 2:
        operations.append("swap")
    operation = rng.choice(operations)
    package_id = rng.choice(package_ids)

    # setelah rng, langsung cek action
    if operation == "swap":
        first_id, second_id = rng.sample(package_ids, 2)
        candidate = try_swap(original, first_id, second_id)
    elif operation == "rotate":
```

```
        candidate = try_rotate(original, package_id, rng.choice(("x",
"y", "z")))

    # kalau bukan rotate dan swap, lakukan move
    else:
        if package_id not in original.out_pkg and rng.random() < 0.2:
            candidate = try_move(original, package_id, None)
        else:
            position = random_position(original, package_id, rng)
            if position is None:
                return original
            candidate = try_move(original, package_id, position)

    if candidate is None:
        return original

    return candidate
```

Fungsi mutation berperan untuk memberi pergerakan genetik baru pada individu. Tujuannya mengeksplorasi ruang pencarian yang belum tersentuh oleh parent. Mirip dengan pencarian neighbor pada algoritma Local Search lain, mutation memilih satu aksi modifikasi secara acak. Aksi ini bisa berupa swap letak dua paket, rotate orientasi paket, atau move paket ke koordinat baru di dalam truk. Jika modifikasi ini melanggar constraints, aksinya langsung dibatalkan. Individu akan kembali utuh ke bentuk aslinya. Sebaliknya, jika kandidat tersebut valid, konfigurasi child akan berubah secara permanen.

2.3.4 Algoritma Bonus

Selain implementasi algoritma pencarian standar, tim juga mengembangkan dua algoritma tambahan yang merupakan variasi dari metode Hill-Climbing, yakni Hill-Climbing with Sideways Move dan Stochastic Hill-Climbing. Pendekatan Hill-Climbing konvensional memiliki kelemahan karena pencarian akan langsung berhenti ketika terjebak pada local optima. Oleh karena itu, kedua variasi ini dirancang untuk memberikan mekanisme alternatif dalam menghindari kebuntuan tersebut, sehingga algoritma memiliki keleluasaan lebih untuk mengeksplorasi ruang pencarian dan memperbesar peluang menemukan global optimum.

2.3.4.1 Algoritma Bonus Hill Climbing with Sideways Move

Algoritma Hill Climbing with Sideways Move adalah variasi dari algoritma pencarian lokal Hill Climbing standar yang dirancang untuk mengatasi masalah flat local maximum atau area datar. Pada Hill Climbing biasa, proses pencarian akan langsung berhenti jika algoritma tidak menemukan tetangga (neighbor) yang memiliki nilai lebih tinggi dari keadaan saat ini. Variasi ini mengatasi kebuntuan tersebut dengan mengizinkan algoritma untuk bergerak mendatar (sideways move) menuju tetangga yang memiliki nilai (value) persis sama dengan keadaan saat ini. Tujuannya adalah

menjelajahi area datar tersebut dengan harapan menemukan titik baru untuk kembali mendaki ke arah global optimum. Untuk mencegah algoritma berputar-putar tanpa batas di dataran yang sama, diterapkan batasan melalui parameter max_sideways_moves.

src/algo/hill_climb_sideway.py

```
def hill_climbing_with_sideways_move(
    initial_state: State,
    *,
    max_sideways_moves: int = 100,
    seed: int | None = None,
    on_iteration: Callable[[State, int], None] | None = None,
) -> Result:
    if isinstance(max_sideways_moves, bool) or not
isinstance(max_sideways_moves, int):
        raise TypeError("max_sideways_moves harus berupa integer")
    if max_sideways_moves < 0:
        raise ValueError("max_sideways_moves tidak boleh negatif")

    started_at = perf_counter()
    current = initial_state.copy()
    if not Evaluate.is_valid(current):
        raise ValueError("initial_state harus valid")

    current.value = Evaluate.value(current)
    initial = current.copy()
    value_history = [current.value]
    iterations = 0
    sideways_moves = 0
    rng = Random(seed)

    while True:
        best_value = current.value
        best_neighbors: list[State] = []
        for candidate in neighbors(current):
            if candidate.value > best_value:
                best_value = candidate.value
                best_neighbors = [candidate]
            elif candidate.value == best_value:
                best_neighbors.append(candidate)

        iterations += 1
        if not best_neighbors or (
            best_value == current.value and sideways_moves >=
max_sideways_moves
        ):
            value_history.append(current.value)
            if on_iteration is not None:
                on_iteration(current.copy(), iterations)
            break

        best_neighbor = rng.choice(best_neighbors)
        if best_neighbor.value == current.value:
            sideways_moves += 1
        current = best_neighbor
```

```
        value_history.append(current.value)
        if on_iteration is not None:
            on_iteration(current.copy(), iterations)
        if max_sideways_moves > 0 and sideways_moves >=
max_sideways_moves:
            break

    return Result(
        initial_state=initial,
        state=current.copy(),
        value_history=value_history,
        iterations=iterations,
        duration_seconds=perf_counter() - started_at,
        metrics={
            "sideways_moves": sideways_moves,
            "max_sideways_moves": max_sideways_moves,
        },
    )
```

Fungsi hill_climbing_with_sideways_move bertindak sebagai pengontrol dari. Proses dimulai dengan memvalidasi input batas pergerakan max_sideways_moves dan memastikan initial_state valid. Algoritma kemudian mengevaluasi nilai awal dari state tersebut dan menyiapkan variabel pelacakan seperti jumlah iterasi dan jumlah pergerakan mendatar yang telah dilakukan (sideways_moves).

Di dalam perulangan utama, algoritma mengevaluasi seluruh kandidat tetangga yang dapat dijangkau dari state saat ini menggunakan utilitas neighbors. Selama evaluasi, algoritma selalu mencari tetangga dengan nilai tertinggi. Jika ditemukan kandidat yang nilainya memecahkan rekor nilai terbaik sementara, daftar tetangga terbaik (best_neighbors) akan dikosongkan dan diisi hanya dengan kandidat baru tersebut. Namun, jika kandidat memiliki nilai yang setara dengan rekor terbaik, kandidat tersebut cukup ditambahkan ke dalam daftar. Mekanisme ini memungkinkan algoritma mengumpulkan beberapa opsi pergerakan sekaligus, baik untuk mendaki naik maupun bergerak mendatar.

Setelah semua tetangga dievaluasi, algoritma memeriksa kondisi pemberhentian. Pencarian akan langsung dihentikan jika tidak ada tetangga yang tersedia, atau jika nilai tetangga terbaik ternyata sama dengan nilai saat ini dan batas max_sideways_moves sudah habis. Jika kondisi masih aman, algoritma akan memilih satu tetangga secara acak dari daftar terbaik yang sudah dikumpulkan. Apabila tetangga terpilih ternyata tidak memberikan peningkatan nilai, metrik sideways_moves akan bertambah satu. Keadaan saat ini kemudian diganti dengan tetangga terpilih, dan siklus terus berlanjut hingga algoritma menemukan puncak tertinggi.

2.3.4.1 Algoritma Bonus Hill Climbing

Algoritma Random Restart Hill Climbing adalah metode yang dibangun di atas algoritma Hill Climbing standar. Pendekatan ini mengatasi masalah local optima dengan menjalankan ulang proses pencarian beberapa kali dari titik awal yang berbeda. Saat algoritma mencapai puncak lokal dan berhenti, sistem akan melakukan restart

menggunakan initial state acak yang baru. Dari seluruh sesi pencarian tersebut, algoritma menyimpan hasil dengan value tertinggi sebagai solusi akhir.

src/algo/hill_climb_random_restart.py

```
def random_restart_hill_climbing(
    initial_state: State,
    *,
    max_restarts: int = 10,
    seed: int | None = None,
    on_iteration: Callable[[State, int], None] | None = None,
) -> Result:
    if isinstance(max_restarts, bool) or not isinstance(max_restarts,
int):
        raise TypeError("max_restarts harus berupa integer")
    if max_restarts < 0:
        raise ValueError("max_restarts tidak boleh negatif")

    started_at = perf_counter()
    initial = initial_state.copy()
    if not Evaluate.is_valid(initial):
        raise ValueError("initial_state harus valid")
    rng = Random(seed)
    best = initial.copy()
    iterations = 0
    value_history = [initial.value]
    iterations_per_restart: list[int] = []
    restart_histories: list[list[int]] = []

    def report(state: State, iteration: int) -> None:
        if on_iteration is not None:
            on_iteration(state, iterations + iteration)

    for restart in range(max_restarts + 1):
        current = initial if restart == 0 else
random_initial_state(initial.problem, rng)
        result = steepest_ascent_hill_climbing(
            current, on_iteration=report if on_iteration is not None
else None
        )
        iterations += result.iterations
        iterations_per_restart.append(result.iterations)
        restart_histories.append(result.value_history)
        value_history.extend(result.value_history[1:])
        if result.state.value > best.value:
            best = result.state.copy()

    return Result(
        initial_state=initial,
        state=best,
        value_history=value_history,
        iterations=iterations,
        duration_seconds=perf_counter() - started_at,
        metrics={
            "restart_count": max_restarts,
```

```
            "max_restarts": max_restarts,
            "iterations_per_restart": iterations_per_restart,
            "restart_histories": restart_histories,
        },
    )
```

Fungsi random_restart_hill_climbing bertindak sebagai pengontrol perulangan restart. Proses diawali dengan validasi parameter max_restarts dan inisialisasi variabel pelacakan seperti total iterasi dan riwayat value. Inti dari algoritma ini terletak pada perulangan yang berjalan sebanyak max_restarts + 1 kali. Pada putaran pertama, algoritma menjalankan utilitas steepest_ascent_hill_climbing menggunakan initial_state bawaan. Pada putaran berikutnya, fungsi selalu membangkitkan konfigurasi penempatan acak baru menggunakan random_initial_state sebagai titik awal pendakian. Selesai satu sesi Hill Climbing, fungsi mencatat jumlah iterasi yang dihabiskan dan menggabungkan metrik riwayat value. Jika hasil pencarian pada sesi tersebut melampaui rekor best sementara, state terbaik akan langsung diperbarui. Siklus ini terus berlanjut sampai kuota restart habis. Fungsi pada akhirnya mengembalikan konfigurasi truk paling optimal dari keseluruhan percobaan.

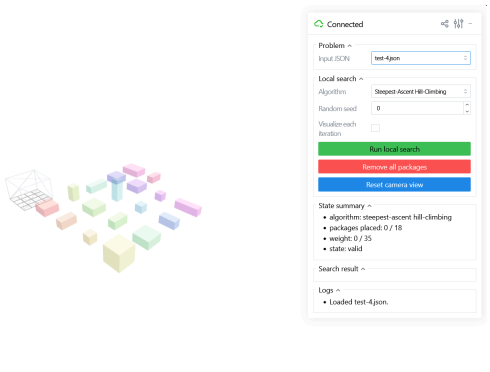
2.4 Antarmuka dan Visualisasi

Pada sistem kami terdapat dua pilihan visualisasi. Pertama, sistem menggunakan viser untuk visualisasi initial state serta final state dari hasil algoritma. Pada tampilan muka, terdapat opsi untuk menentukan tampilan *initial state* ataupun *final state*. Selain itu, terdapat state summary yang berisi algorithm, displayed state, value, packages placed, weight, dan status valid state. Untuk menjalankan ini, user harus menjalankan command ‘uv run solve’ terlebih dahulu untuk mendapatkan file input json state yang sesuai, sehingga dapat divisualisasikan oleh ‘uv run vis-state’.



Gambar 2.4.1 Visualiser untuk Initial dan Final State

Kedua, sistem menggunakan viser untuk visualisasi seluruh proses pencarian dari awal sampai akhir. Pilihan ini dapat digunakan untuk melihat seluruh step satu per satu, tapi tidak memunculkan snapshot initial state. Disini user dapat memilih problem, algoritma, dan parameter-parameternya.



Gambar 2.4.2 Visualiser dan GUI

2.5 Hasil dan Pembahasan

Bagian ini menyajikan evaluasi dari eksperimen implementasi algoritma Hill-Climbing, Simulated Annealing, dan Genetic Algorithm untuk pemecahan persoalan 3D knapsack. Rincian data kuantitatif yang diperoleh selama pengujian mencakup capaian nilai objective function akhir, durasi pencarian komputasi, metrik spesifik dari tiap algoritma, beserta visualisasi plot iterasi dan hasil visualisasi pengujian. Data-data tersebut kemudian dikaji untuk membandingkan performa antar algoritma, mengevaluasi tingkat kedekatan solusi dengan global optima, meninjau konsistensi pencarian, serta menganalisis bagaimana pengaruh perubahan parameter memengaruhi konfigurasi akhir yang dihasilkan.

2.5.1 Hasil Eksperimen

Bagian ini menyajikan hasil dari seluruh eksperimen yang telah dilakukan terhadap setiap algoritma pencarian. Pengujian dilakukan dengan menjalankan masing-masing algoritma menggunakan konfigurasi parameter dan *seed* awal yang berbeda untuk memastikan objektivitas pengamatan. Data yang dihimpun dari setiap percobaan meliputi kondisi *state* awal dan akhir, peningkatan nilai *objective function*, durasi komputasi, serta rasio paket yang berhasil dimuat ke dalam truk.

2.5.1.1 Eksperimen Steepest-Ascent Hill Climbing

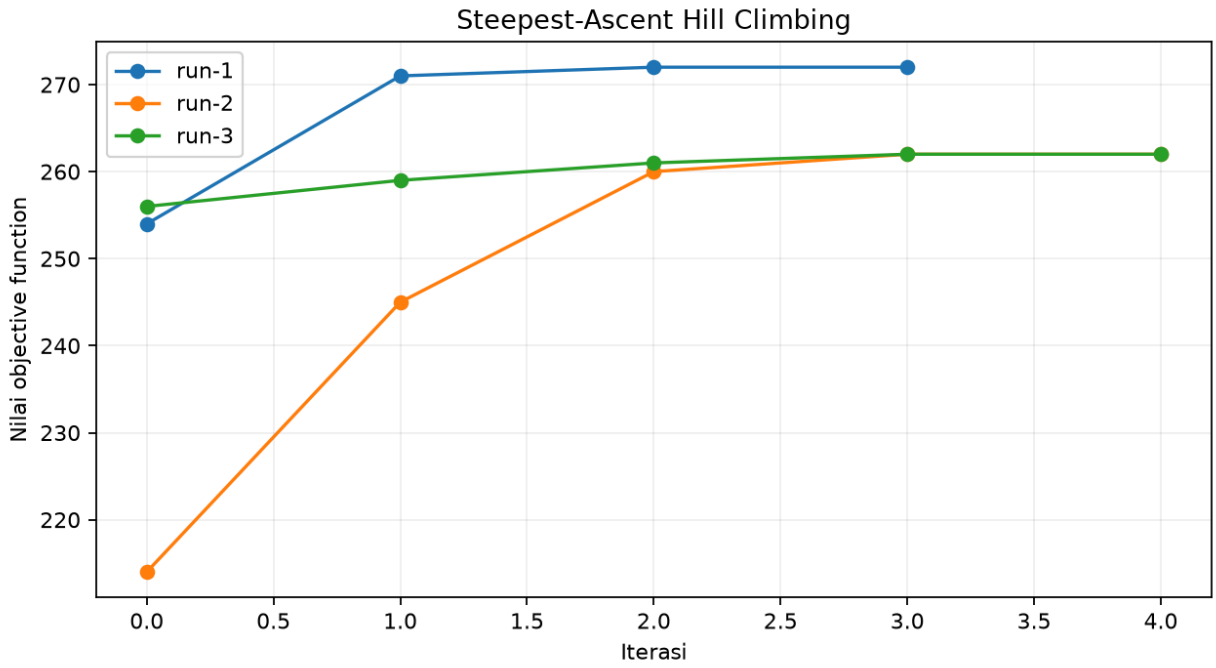
Eksperimen dilakukan menggunakan input test-4.json dengan tiga seed berbeda, yaitu 100, 101, dan 102. Input terdiri atas 18 paket, truk berdimensi 4 x 4 x 3, dan kapasitas maksimum 35 unit berat. Setiap run menggunakan initial state acak yang dibangkitkan berdasarkan seed masing-masing.

Tabel 2.5.1.1.1 Tabel Hasil Eksperimen Steepest-Ascent Hill Climbing

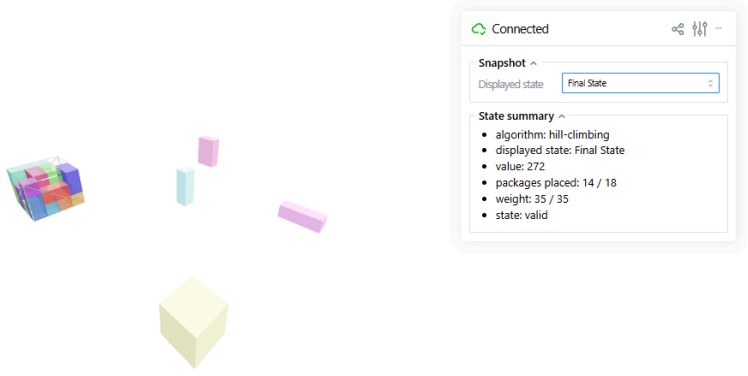
Run	Seed	Nilai awal	Nilai akhir	Perubahan nilai	Jumlah state yang di cek	Durasi	Paket dalam truk
1	100	254	272	18	5	0.068215	14 / 18

2	101	214	262	48	20	0.167688	13 / 18
3	102	256	262	6	7	0.197518	12 / 18

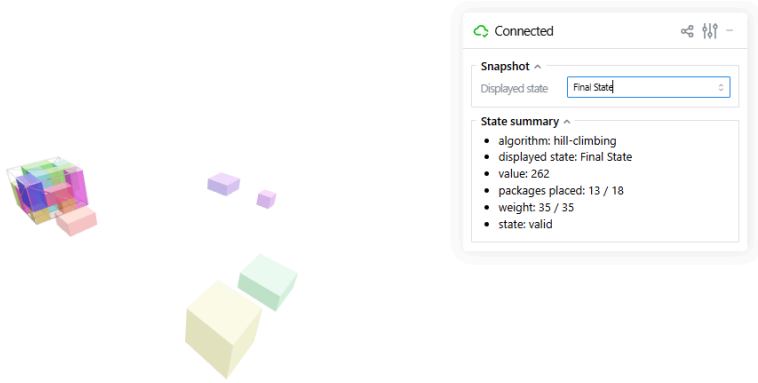
Berikut adalah plot value dari objective function.



Gambar 2.5.1.1.1 Gambar plot value Steepest-Ascent Hill Climbing (gambar bukan hasil dari program melainkan secara manual menggabungkan ke-3 run)



Gambar 2.5.1.1.2 Gambar visualisasi Steepest-Ascent Hill Climbing run-1



Gambar 2.5.1.1.2 Gambar visualisasi Steepest-Ascent Hill Climbing run-2



Gambar 2.5.1.1.2 Gambar visualisasi Steepest-Ascent Hill Climbing run-3

2.5.1.2 Eksperimen Simulated Annealing

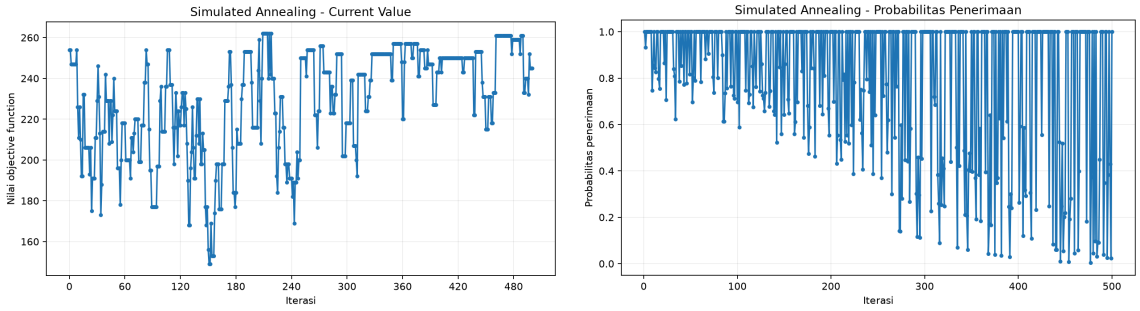
Eksperimen dilakukan menggunakan input test-4.json dengan jumlah iterasi 500 dengan tiga seed berbeda, yaitu 100, 101, dan 102. Input terdiri atas 18 paket, truk berdimensi 4 x 4 x 3, dan kapasitas maksimum 35 unit berat. Setiap run menggunakan initial state acak yang dibangkitkan berdasarkan seed masing-masing.

Tabel 2.5.1.2.1 Tabel Hasil Eksperimen Simulated Annealing

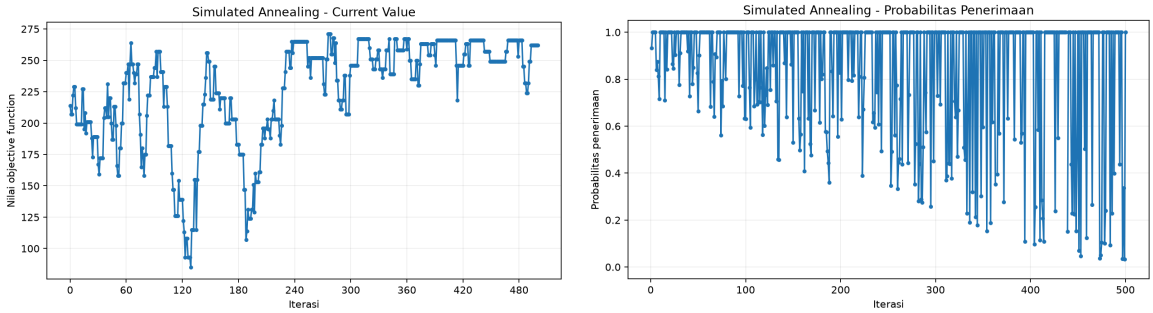
Run	Seed	Nilai awal	Nilai akhir	Perubahan nilai	Jumlah iterasi yang diterima	Jumlah stuck	Durasi	Paket dalam truk
1	100	254	262	8	394	106	1.498007	14 / 18
2	101	214	271	57	414	86	1.216588	13 / 18

3	102	256	272	16	403	97	1.212233	13 / 18

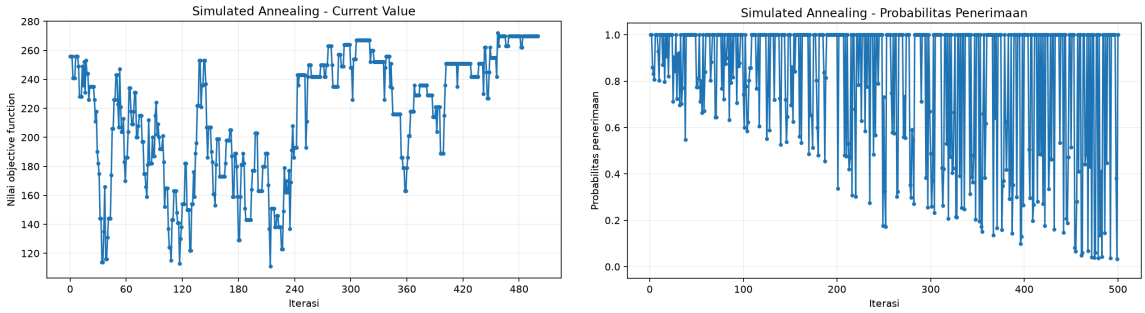
Berikut adalah plot value & frekuensi stuck dari objective function:



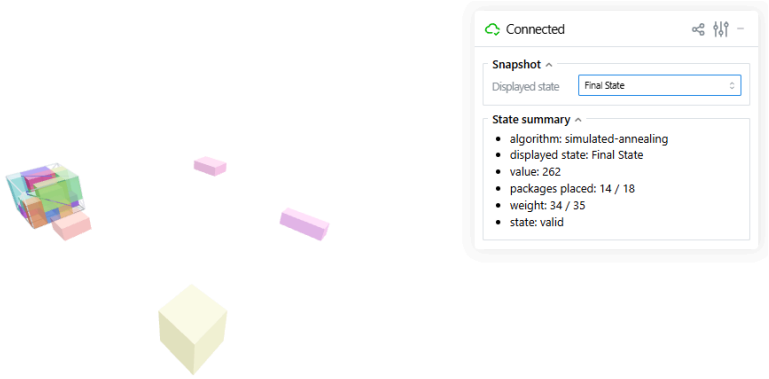
Gambar 2.5.1.2.1 Gambar plot value & frekuensi stuck Simulated Annealing seed 100



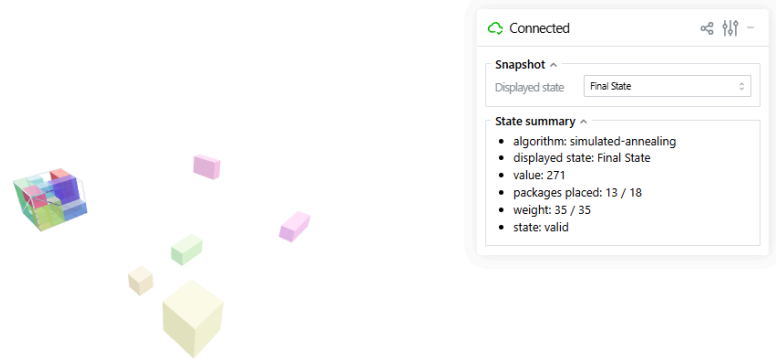
Gambar 2.5.1.2.2 Gambar plot value & frekuensi stuck Simulated Annealing seed 101



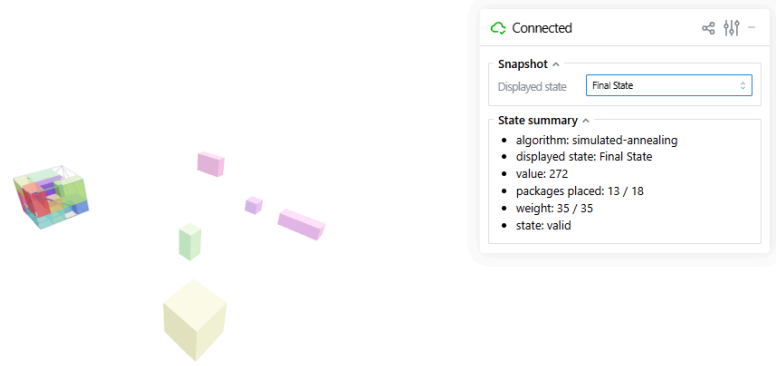
Gambar 2.5.1.2.3 Gambar plot value & frekuensi stuck Simulated Annealing seed 102



Gambar 2.5.1.2.4 Gambar visualisasi Simulated Annealing run ke-1



Gambar 2.5.1.2.5 Gambar visualisasi Simulated Annealing run ke-2



Gambar 2.5.1.2.6 Gambar visualisasi Simulated Annealing run ke-3

2.5.1.3 Eksperimen Genetic Algorithm

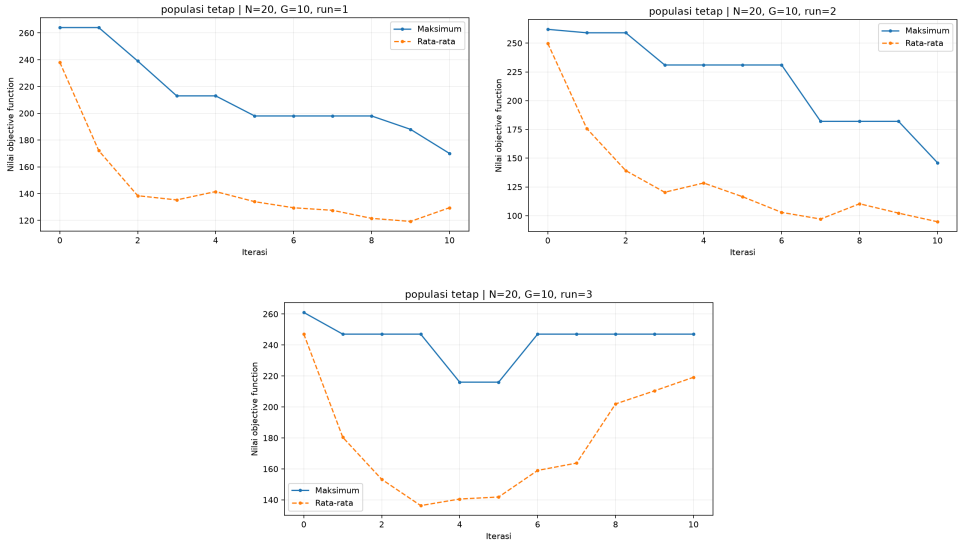
Eksperimen dilakukan menggunakan input test-4.json dengan seed yang berbeda-beda. Input terdiri atas 18 paket, truk berdimensi 4 x 4 x 3, dan kapasitas maksimum 35 unit berat. Setiap run menggunakan initial state acak yang dibangkitkan berdasarkan seed masing-masing.

Eksperimen dilakukan dengan menggunakan jumlah populasi 20 sebagai kontrol untuk tiga jumlah iterasi yang berbeda dan menggunakan iterasi 30 untuk tiga jumlah populasi yang berbeda.

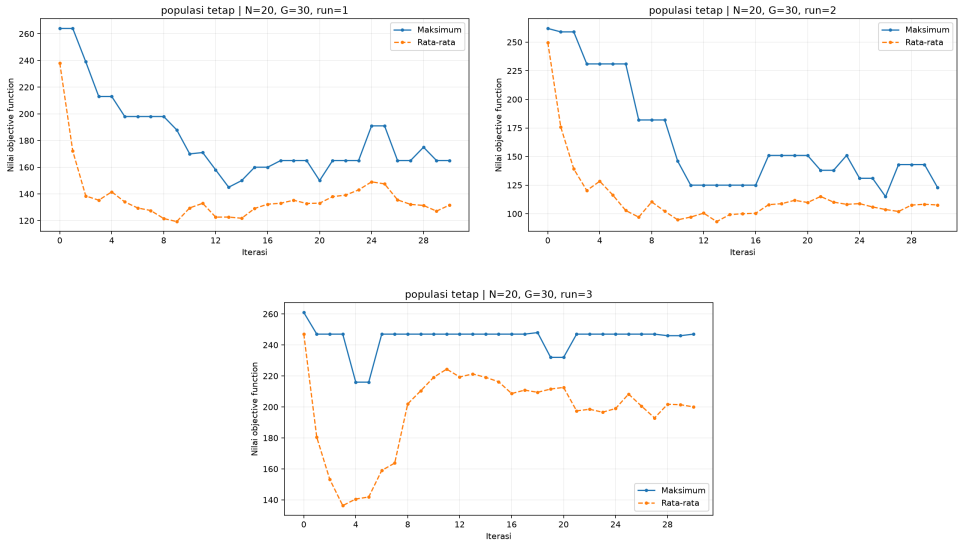
Tabel 2.5.1.3.1 Tabel Hasil Eksperimen Genetic Algorithm

Kelompok	Konfigurasi	Run	Seed	Nilai awal	Nilai akhir	Perubahan nilai	Jumlah Populasi	Banyak Iterasi	Durasi	Paket dalam truk
Populasi Tetap	1	1	1	264	170	-94	20	10	1.60	7 / 18
		2	2	262	146	-116	20	10	1.50	6 / 18
		3	3	261	247	-14	20	10	1.66	9 / 18
	2	1	1	264	165	-99	20	30	2.80	6 / 18
		2	2	262	123	-139	20	30	2.49	6 / 18
		3	3	261	247	-14	20	30	3.41	9 / 18
	3	1	1	264	182	-82	20	50	3.85	7 / 18
		2	2	262	160	-102	20	50	3.43	6 / 18
		3	3	261	247	-14	20	50	5.13	9 / 18
Iterasi Tetap	4	1	4	259	128	-131	10	30	1.36	5 / 18
		2	5	262	122	-140	10	30	1.15	5 / 18
		3	6	262	211	-51	10	30	2.01	11 / 18
	5	1	4	263	181	-82	20	30	2.74	7 / 18
		2	5	266	168	-98	20	30	2.37	7 / 18
		3	6	267	184	-83	20	30	3.01	9 / 18
	6	1	4	269	225	-44	30	30	5.36	10 / 18
		2	5	268	218	-50	30	30	5.51	11 / 18
		3	6	267	171	-96	30	30	3.85	7 / 18

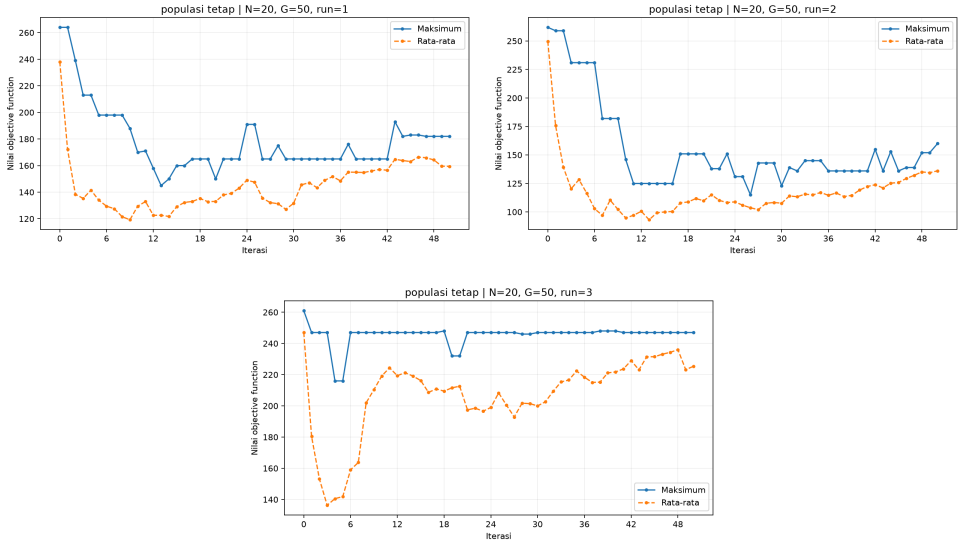
Berikut adalah plot value dari objective function



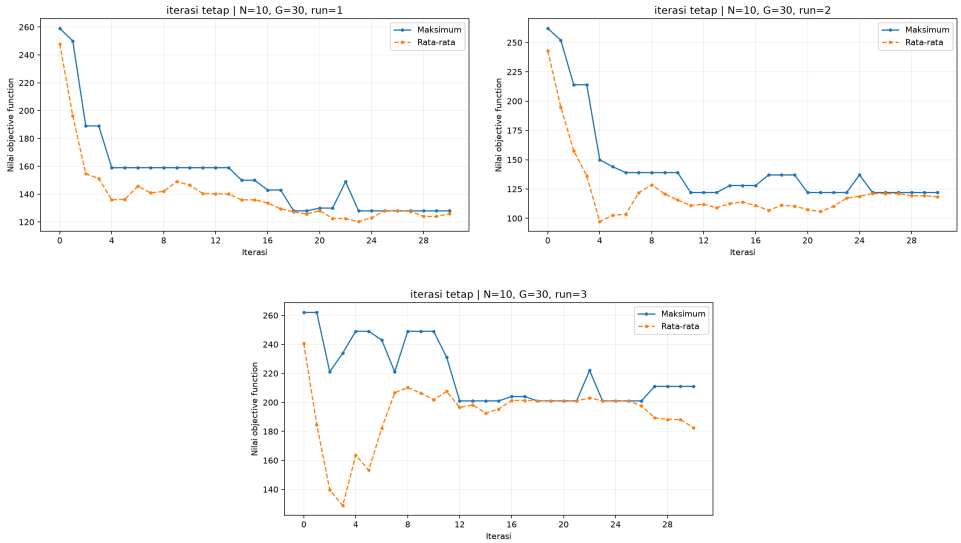
Gambar 2.5.1.3.1 Gambar plot value Genetic Algorithm Konfigurasi 1



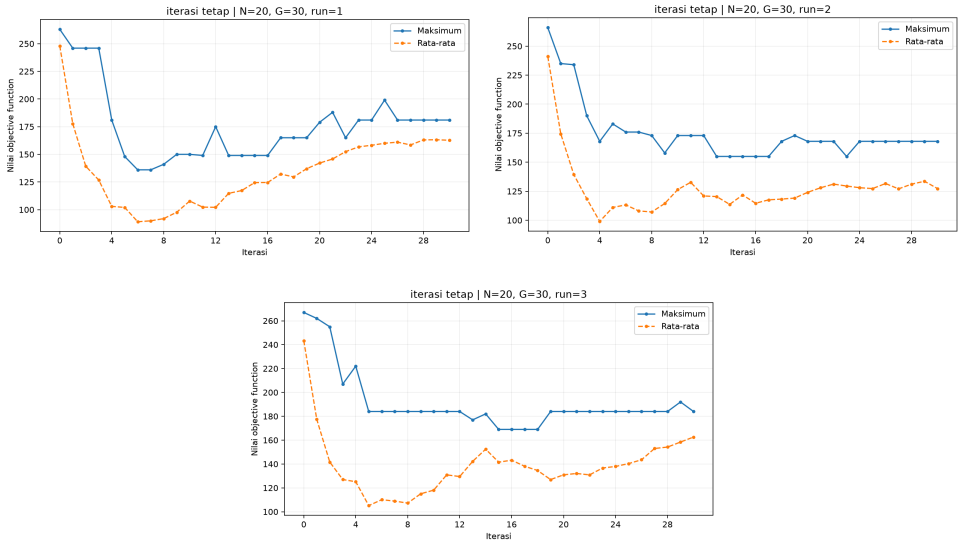
Gambar 2.5.1.3.2 Gambar plot value Genetic Algorithm Konfigurasi 2



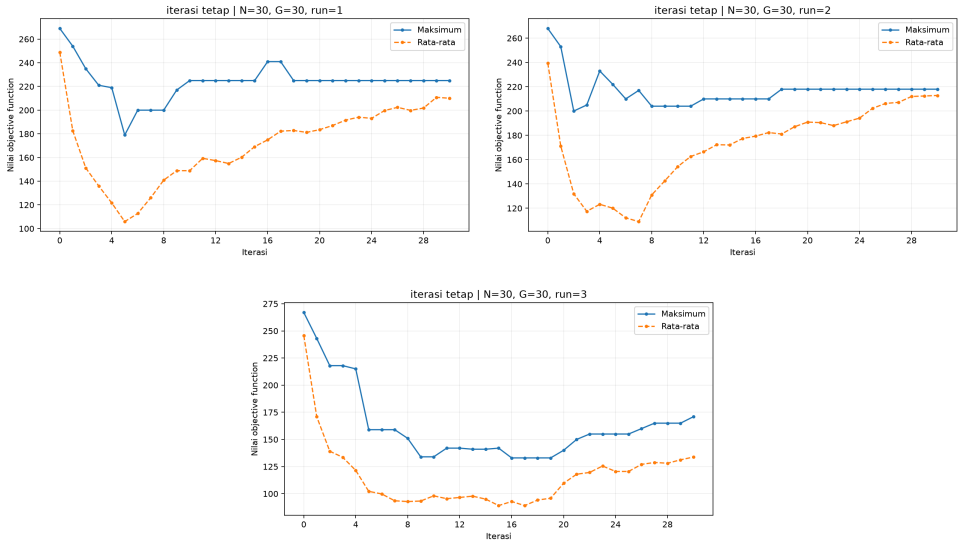
Gambar 2.5.1.3.3 Gambar plot value Genetic Algorithm Konfigurasi 3



Gambar 2.5.1.3.4 Gambar plot value Genetic Algorithm Konfigurasi 4



Gambar 2.5.1.3.5 Gambar plot value Genetic Algorithm Konfigurasi 5



Gambar 2.5.1.3.6 Gambar plot value Genetic Algorithm Konfigurasi 6

2.5.1.4 Eksperimen Hill Climbing with Sideways Move

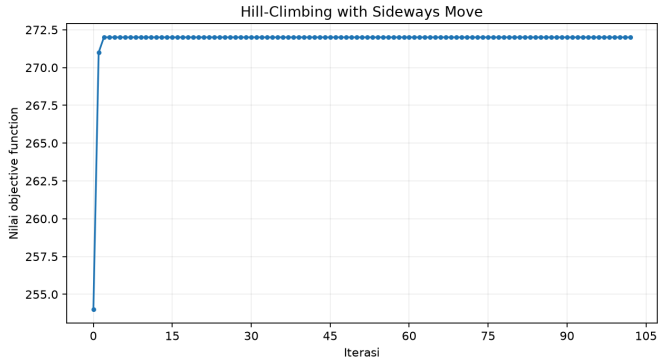
Eksperimen dilakukan menggunakan input test-4.json dengan tiga seed berbeda, yaitu 100, 101, dan 102. Input terdiri atas 18 paket, truk berdimensi 4 x 4 x 3, dan kapasitas maksimum 35 unit berat. Setiap run menggunakan initial state acak yang dibangkitkan berdasarkan seed masing-masing.

Tabel 2.5.1.4.1 Tabel Hasil Eksperimen Hill Climbing with Sideways Move

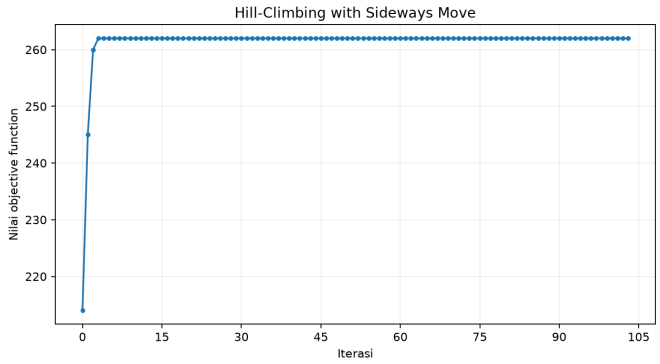
Run	Seed	Nilai awal	Nilai akhir	Perubahan	Jumlah state	Durasi	Paket

				nilai	yang di cek		dalam truk
1	100	254	272	18	102	16.164960	14 / 18
2	101	214	262	48	103	28.264294	13 / 18
3	102	256	262	6	103	26.859439	12 / 18

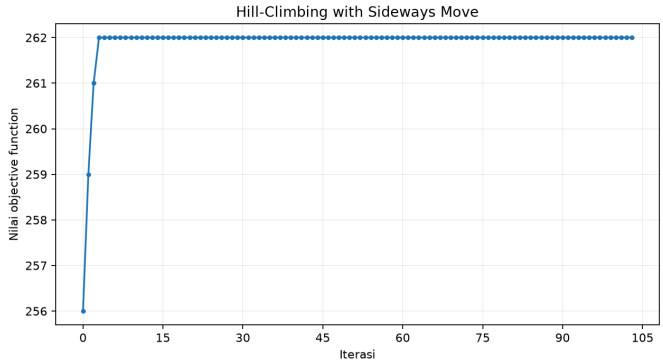
Berikut adalah plot value dari objective function.



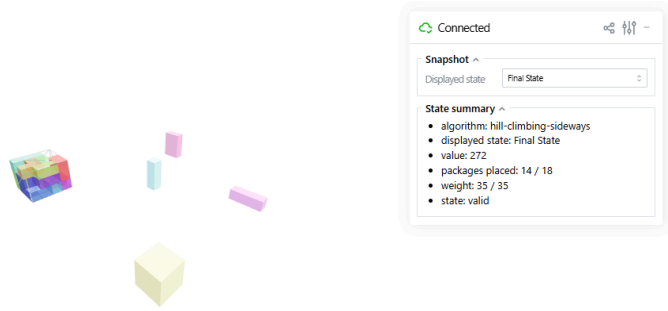
Gambar 2.5.1.4.1 Gambar plot value Hill Climbing with Sideways Move run ke-1



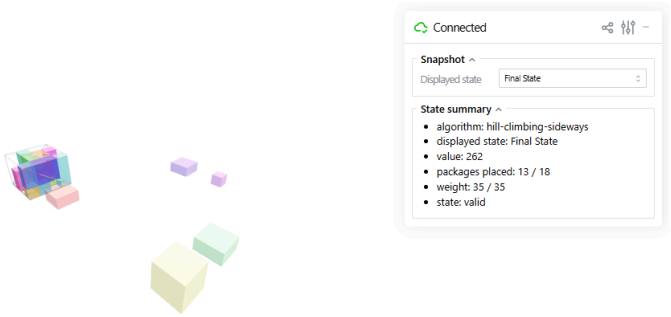
Gambar 2.5.1.4.2 Gambar plot value Hill Climbing with Sideways Move run ke-2



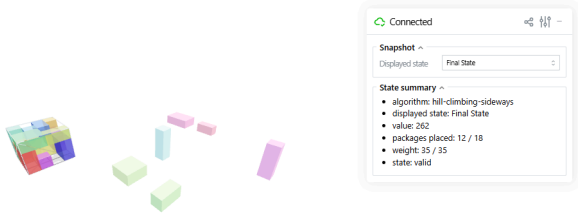
Gambar 2.5.1.4.3 Gambar plot value Hill Climbing with Sideways Move run ke-3



Gambar 2.5.1.4.4 Gambar visualisasi Hill Climbing with Sideways Move run-1



Gambar 2.5.1.4.5 Gambar visualisasi Hill Climbing with Sideways Move run-2



Gambar 2.5.1.4.6 Gambar visualisasi Hill Climbing with Sideways Move run-3

2.5.1.5 Eksperimen Random Restart Hill Climbing

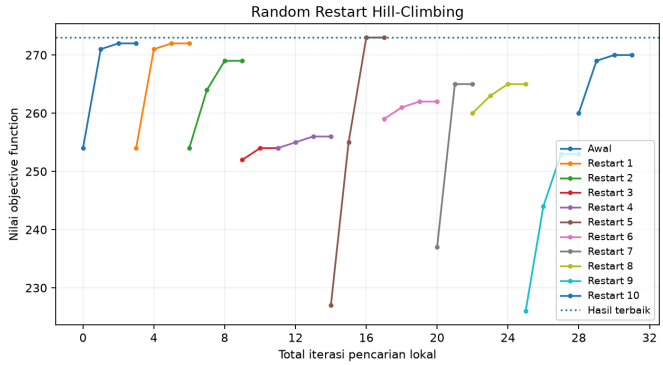
Eksperimen dilakukan menggunakan input test-4.json dengan tiga seed berbeda, yaitu 100, 101, dan 102. Input terdiri atas 18 paket, truk berdimensi 4 x 4 x 3, dan kapasitas maksimum 35 unit berat. Setiap run menggunakan initial state acak yang dibangkitkan berdasarkan seed masing-masing.

Tabel 2.5.1.5.1 Tabel Hasil Eksperimen Random Restart Hill Climbing

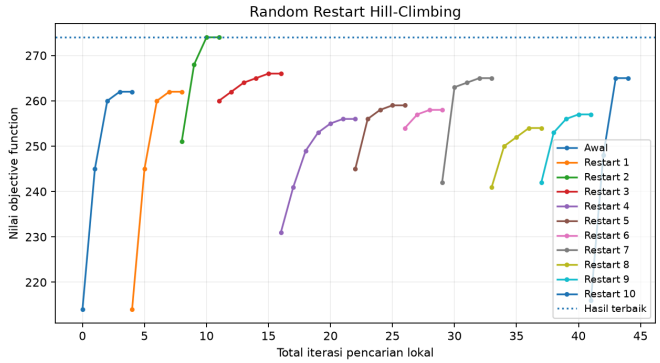
Run	Seed	Nilai awal	Nilai akhir	Perubahan	Jumlah state	Durasi	Paket

				nilai	yang di cek		dalam truk
1	100	254	273	19	31	6.681866	13 / 18
2	101	214	274	48	44	9.848008	14 / 18
3	102	256	272	16	36	6.398519	14 / 18

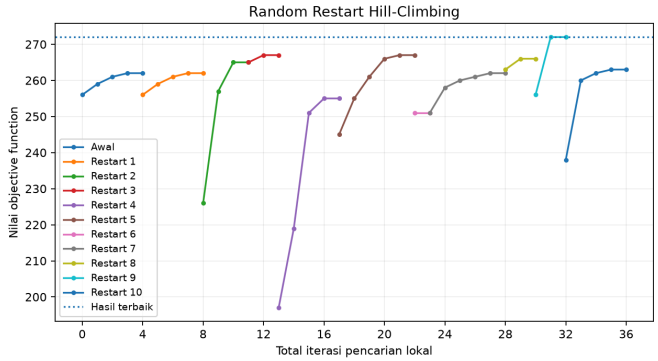
Berikut adalah plot value dari objective function.



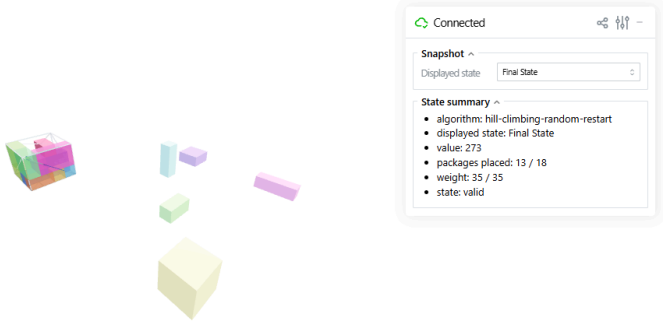
Gambar 2.5.1.5.1 Gambar plot value Random Restart Hill Climbing run ke-1



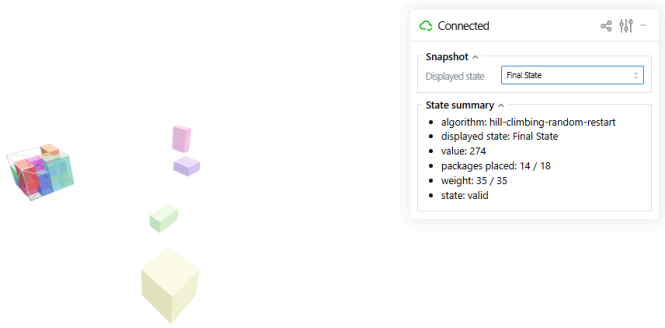
Gambar 2.5.1.5.2 Gambar plot value Random Restart Hill Climbing run ke-2



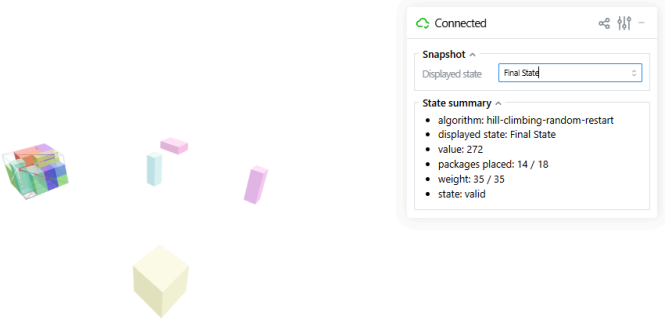
Gambar 2.5.1.5.3 Gambar plot value Random Restart Hill Climbing run ke-3



Gambar 2.5.1.5.4 Gambar visualisasi Random Restart Hill Climbing run-1



Gambar 2.5.1.5.5 Gambar visualisasi Random Restart Hill Climbing run-2



Gambar 2.5.1.5.6 Gambar visualisasi Random Restart Hill Climbing run-3

2.5.2 Pembahasan

- Seberapa dekat tiap-tiap algoritma bisa mendekati global optima dan mengapa hasilnya demikian?

  Pada hasil eksperimen sebelumnya, nilai tertinggi ditemukan menggunakan Steepest-Ascent Hill sekaligus Simulated Annealing, menghasilkan value 272, dengan seed 100 untuk Steepest-Ascent dan seed 102 untuk Simulated Annealing. Genetic Algorithm menghasilkan nilai akhir tertinggi 247 pada kelompok populasi tetap. Bisa dikatakan bahwa Genetic Algorithm merupakan algoritma yang lemah dalam pendekatan global optima.

Steepest-Ascent Hill Climbing memperoleh hasil melalui perpindahan yang selalu meningkatkan nilai, tetapi dapat berhenti ketika tidak menemukan tetangga yang lebih baik. Simulated Annealing memungkinkan penerimaan langkah yang menurunkan nilai sehingga dapat mengeksplorasi konfigurasi lain. Pada Genetic Algorithm, individu terbaik tidak otomatis dipertahankan karena tidak ada mekanisme yang mempertahankan individu terbaik, misal seperti dengan menggunakan elitisme, sedangkan hasil yang dikembalikan adalah individu terbaik dari populasi terakhir. Selain itu, kode crossover mengabaikan paket warisan yang gagal ditempatkan secara valid. Mekanisme tersebut dapat menjelaskan kemungkinan penurunan kualitas Genetic Algorithm.

- Bagaimana perbandingan hasil pencarian tiap-tiap algoritma dengan algoritma local search yang lain?

  Simulated Annealing memberikan hasil rerata yang lebih bagus pada ketiga run dibanding Steepest-Ascent Hill Climbing. Hal ini bisa terlihat pada Tabel 2.5.1.1.1 dan Tabel 2.5.1.2.1, di mana Steepest-Ascent Hill Climbing menghasilkan rerata 265,33 dan Simulated Annealing menghasilkan rerata 268,33. Sementara itu, Genetic Algorithm menghasilkan nilai akhir antara 122 dan 247, sehingga seluruh hasil akhirnya berada di bawah kedua algoritma lainnya. Seluruh run Genetic Algorithm juga mengalami penurunan dibandingkan individu terbaik populasi awal, dengan perubahan nilai antara −140 dan −14. Hasil ini menunjukkan bahwa pada konfigurasi yang diuji, proses pergantian generasi Genetic Algorithm belum berhasil mempertahankan kualitas solusi awal. Meskipun demikian, perbandingan antara algoritma belum sepenuhnya setara karena Genetic Algorithm menggunakan seed, populasi awal, dan anggaran pencarian yang berbeda.

- Bagaimana perbandingan durasi proses pencarian tiap algoritma relatif terhadap algoritma lainnya?

  Steepest-Ascent Hill Climbing merupakan algoritma tercepat pada eksperimen yang dicatat, dengan durasi rerata sekitar 0,144 detik. Sementara itu, Simulated Annealing membutuhkan rerata sekitar 1,309 detik, atau sekitar 9 kali rerata durasi Steepest-Ascent Hill Climbing. Pada Genetic Algorithm, dengan populasi tetap 20, rerata durasi meningkat dari 1,59 detik untuk 10 generasi menjadi 2,90 detik untuk 30 generasi dan 4,14 detik untuk 50 generasi (dalam rata-rata). Pada kelompok iterasi tetap 30, rerata durasinya meningkat dari 1,51 detik untuk populasi 10 menjadi 2,71 detik untuk populasi 20 dan 4,91 detik untuk populasi 30 (dalam rata-rata). Bertambahnya populasi dan generasi menambah jumlah individu yang harus dibentuk serta dievaluasi, tetapi tambahan waktu tersebut belum menghasilkan kualitas yang melampaui kedua algoritma lainnya pada pengujian ini akibat dari kompleksitas model yang menyusahkan proses crossover.Susunan yang valid pada masing-masing parent belum tentu tetap valid ketika digabungkan, karena paket dapat bertabrakan, kehilangan penyangga, atau melanggar batas kapasitas. Pada implementasi yang digunakan, paket warisan yang gagal ditempatkan secara valid dibiarkan berada di luar truk tanpa dicoba pada posisi alternatif, sehingga child dapat kehilangan paket bernilai tinggi dan memiliki fitness lebih rendah daripada parent-nya.

- Seberapa konsisten hasil akhir yang didapatkan dari tiap-tiap eksperimen yang dilakukan?

  Steepest-Ascent Hill Climbing dan Simulated Annealing sama-sama menghasilkan nilai akhir dalam rentang yang cukup konsisten, yakni di antara 262–272. Sementara itu, Genetic Algorithm dengan populasi tetap 20, rentang hasil masing-masing konfigurasi adalah 101 untuk 10 generasi, 124 untuk 30 generasi, dan 87 untuk 50 generasi. Pada kelompok iterasi tetap 30, rentangnya adalah 89 untuk populasi 10, 16 untuk populasi 20, dan 54 untuk populasi 30. Artinya, populasi 20 memberikan hasil paling berdekatan dalam kelompok iterasi tetap, tetapi populasi yang lebih besar tidak selalu menghasilkan konsistensi lebih baik.

- Bagaimana pengaruh banyak iterasi dan jumlah populasi terhadap hasil akhir pencarian pada Genetic Algorithm?

  Pada populasi tetap dengan nilai kontrol 20, rerata nilai akhir berubah dari 187,67 pada 10 iterasi menjadi 178,33 pada 30 iterasi, kemudian meningkat menjadi 196,33 pada 50 iterasi (nilai yang tercantum merupakan nilai rata-rata dari ketiga run). Pola tersebut menunjukkan bahwa penambahan iterasi tidak memberikan perbaikan atau penambahan nilai yang konsisten. Pencarian yang lebih panjang dapat menghasilkan populasi terakhir yang lebih buruk karena tidak ada jaminan individu terbaik bertahan, setidaknya dalam implementasi ini. Sementara itu, pada iterasi tetap 30, peningkatan populasi dari 10 menjadi 20 dan 30 meningkatkan rerata nilai akhir dari 153,67 menjadi 177,67 dan 204,67 (rata-rata dari ketiga run). Yang berarti seiring jumlah populasi bertambah, nilai yang dihasilkan juga semakin membaik. Dengan demikian, populasi lebih besar memperbaiki rerata hasil pada sampel ini dengan biaya waktu lebih tinggi, sedangkan penambahan generasi belum menjamin peningkatan nilai akhir. Hal ini disebabkan karena populasi yang lebih besar memberi lebih banyak peluang menemukan konfigurasi yang baik, sehingga rerata hasil pada eksperimen ini meningkat meskipun tidak pada setiap run.

Untuk algoritma Hill Climbing lainnya, Random Restart Hill Climbing menghasilkan nilai akhir 273, 274, dan 272 dengan rerata 273,00, sehingga memberikan hasil lebih tinggi daripada Sideways Move dan Hill Climbing standar yang sama-sama menghasilkan 272, 262, dan 262 dengan rerata 265,33, serta Simulated Annealing yang memiliki rerata 268,33. Hal yang penting untuk ditandai adalah keunggulan Random Restart. Hal ini bisa bekerja akibat kondisi algoritma yang dapat terus menerus mengulangi keadaan dengan batasan tertentu dari *local optima*, sehingga pencarian bisa disesuaikan kembali agar tidak terlalu cepat dalam melakukan *restart*. Algoritma ini juga tidak sekompleks Genetic Algorithm dalam penentuan suksesornya.

Bab 3

Kesimpulan dan Saran

4.1 Kesimpulan

Implementasi dan evaluasi algoritma *Local Search* untuk memecahkan *3D Bin Packing Problem* membuktikan bahwa setiap variasi heuristik menawarkan karakteristik penyelesaian yang unik dengan *trade-off* yang spesifik antara durasi komputasi dan kedekatan dengan *global optimum*. *Steepest-Ascent Hill Climbing* terbukti sangat cepat tapi mudah terjebak di *local optima* akibat pergerakannya yang murni deterministik. Variasi seperti *Sideways Move* dan *Random Restart* mampu menambal kelemahan tersebut dengan mengeksplorasi area datar atau mengulang pencarian dari titik acak baru, sehingga dapat meraih nilai akhir yang jauh lebih tinggi. Di sisi lain, *Simulated Annealing* memberikan rasio terbaik antara waktu dan kualitas karena kemampuannya mencapai *global optima* secara probabilistik yang diatur oleh penurunan temperatur. Sementara itu, *Genetic Algorithm*, membutuhkan waktu eksekusi paling panjang dan sangat bergantung pada parameter populasi dan iterasi.

4.2 Saran

Sistem masih belum memiliki optimasi maksimal, terutama pada Genetic Algorithm. Untuk pengembangan kedepannya, sistem dapat dimaksimalkan dalam optimasi untuk Genetic Algorithm. Selain itu, masih diperlukan script untuk akses cepat pemakaian serta visualisasi yang mendukung ilustrasi dari tiap algoritma.

Daftar Pustaka

Beyond classical search: IF3170 Inteligensi Artifisial, Semester I Tahun Akademik 2026/2027. (2026). Institut Teknologi Bandung.

Russell, S. J., & Norvig, P. (2010). *Artificial intelligence: A modern approach* (3rd ed.). Prentice Hall.

Pernyataan Penggunaan AI

AI digunakan sebagai alat bantu pembelajaran untuk memahami konsep Intelegensi Artifisial. AI juga digunakan untuk meminta contoh penerapan konsep dan mengklarifikasi istilah yang belum dipahami.